PaDiM Model Optimization

From Research Code to Production-Ready Inference

77x Performance Improvement

March 2026

What is PaDiM & Why Does It Matter?

How It Works:

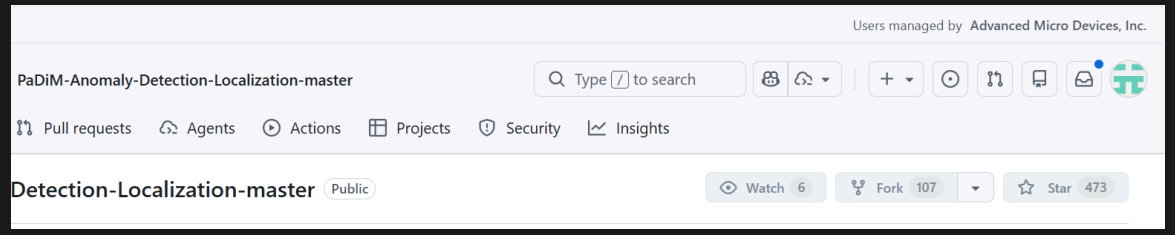
1. **Learn "Normal"**: Extract features from +ve examples using CNN backbone
2. **Build Statistical Model**: Fit Gaussian distribution per patch location
3. **Detect Anomalies**: Measure deviation using Mahalanobis distance

Why PaDiM is Important:

- **Zero-shot anomaly detection** - no -ve samples needed
- **Used in** Manufacturing QC, PCB inspection, Surface anomaly detection
- State-of-the-art - 96%+ AUROC on MVTec AD benchmark

The Challenge:

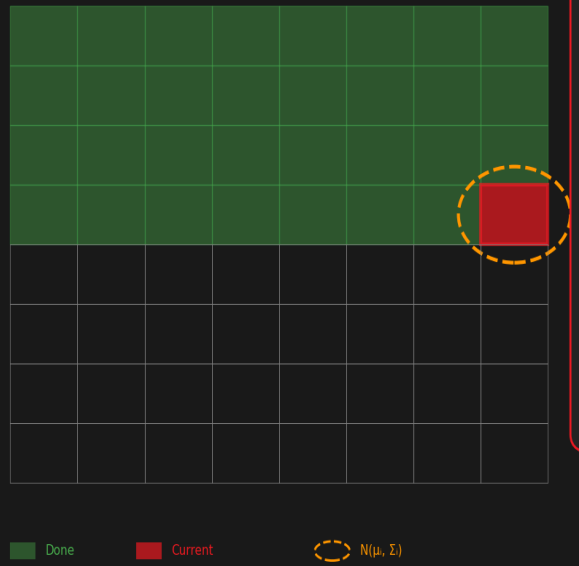
- Implementation from popular open-source repo
- **Not production-ready** out of the box
- **Not designed for edge compute**
- Sequential processing, Python overhead



PaDiM Pipeline: Gaussian Scanning Process

BEFORE: Sequential Loop

Position i = 32 of 3,136



LOOP ALGORITHM

For each position $i = 1, 2, ..., 3136$:

1. Extract feature vector x_i
2. Get learned mean μ_i
3. Get covariance matrix Σ_i
4. **Compute inverse $\Sigma_i^{-1} = \text{inv}(\Sigma_i)$** ← $O(n^3)$
5. Distance $d_i = \sqrt{[(x_i - \mu_i)^T \Sigma_i^{-1} (x_i - \mu_i)]}$

End loop

Inversions computed: **32** of 64

Total operations: **32,000,000**

621 ms per image | 1.6 FPS | TOO SLOW FOR REAL-TIME

Each patch compared against learned Gaussian distribution

Pipeline Steps:

1. Feature Extraction (ResNet18)
2. Embedding Concat (448D)
3. Dimension Reduction (→100D)
4. Gaussian Scan - Compare each patch to learned distribution
5. Anomaly Map - Mahalanobis distance per spatial location

The Bottleneck:

- 3,136 spatial positions
- Each needs distance computation
- Python loop with SciPy calls

The Bottleneck: Where Was Time Being Spent?

Baseline Time Breakdown

Component	Time	% of Total
Feature Extraction	15 ms	2.4%
Embedding Concat	12 ms	1.9%
Mahalanobis Loop	**590 ms**	**95%**
Post-processing	4 ms	0.6%
Total	**621 ms**	100%

Root Cause Analysis

The Problem:

- 3,136 spatial positions to compute
- Each requires 100×100 matrix inverse
- Python loop with SciPy calls

Complexity:

- Matrix inverse: $O(n^3) = O(100^3)$
- Total: 3,136 × $O(100^3)$ operations
- Sequential, non-parallelized

Key Optimization Insight

Inverse Covariance Can Be Pre-computed

Original Approach (Per-Inference):

```
For each of 3,136 positions:
    1. Get covariance matrix Σ
    2. Compute inverse Σ⁻¹  ← SLOW!
    3. Calculate Mahalanobis distance
```

Problem: Computing Σ^{-1} is $O(n^3) = O(100^3)$

Solution:

- Σ^{-1} is **constant** after training
- Compute once, save to disk (~31 MB)
- Load and reuse during inference

Impact: Eliminates 3,136 matrix inversions at runtime

Optimized Pipeline:

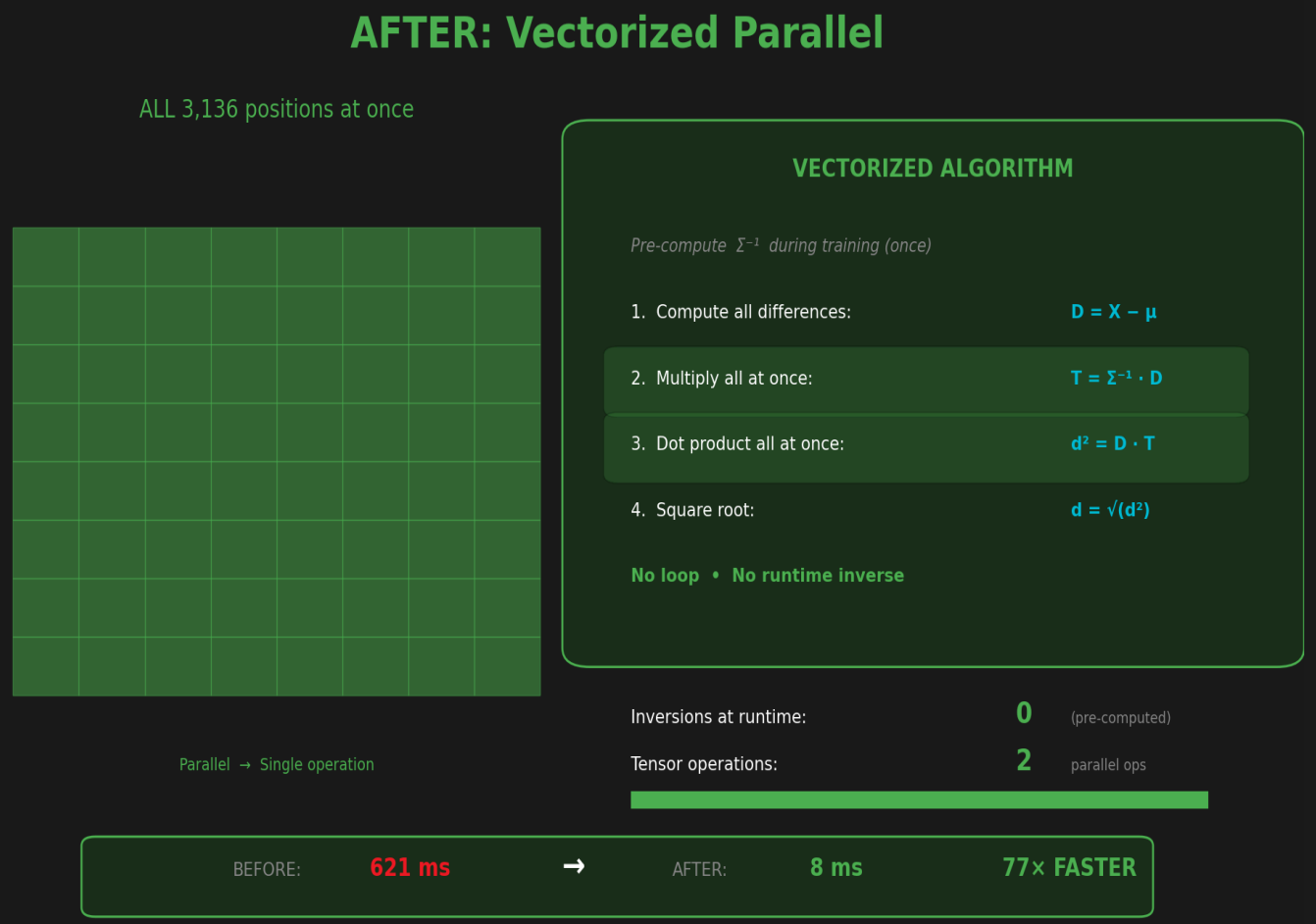
With `inv_cov` and `mean` pre-computed and loaded:

```
# Pre-loaded from training
mean = load("mean.pkl")         # [100, 56, 56]
inv_cov = load("inv_cov.pkl")  # [100, 100, 56, 56]

# Inference - no matrix inverse needed!
diff = embedding - mean
dist = mahalanobis(diff, inv_cov)
```

Result: The expensive operations are moved to training time. Rest of the loop can be **parallelized** and **converted to ONNX**.

Vectorized Mahalanobis Distance



Implementation (Einstein Summation):

```
left = np.einsum('ci,cdij->di', diff, inv_cov)
dist = np.sqrt(np.einsum('di,di->i', diff, left))
```

- **Mahalanobis:** $d(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$
- **Vectorized:** $\text{dist}[i] = \sqrt{\sum_{c,d} \text{diff}[c,i] \cdot \Sigma^{-1}[c,d,i] \cdot \text{diff}[d,i]}$
- **Key Insight:**
  - No Python loops
  - BLAS/LAPACK optimized, CPU SIMD (AVX2)
  - **ONNX convertible**
  - All 3,136 distances in ONE op

Performance Results

Speed Gains from Baseline to Full ONNX GPU

Configuration	Time (ms)	FPS	vs Baseline
Baseline PyTorch CPU (scipy loop)	621.51	1.6	1.0x
Baseline PyTorch GPU (scipy loop)	599.86	1.7	1.1x
+ Pre-computed Σ^{-1}	~80	~12	~8x
+ Vectorized einsum (CPU)	36.22	27.6	17.2x
+ ONNX Runtime (CPU)	15.86	63.1	39.2x
+ GPU (MIGraphX)	**8.01**	**124.8**	**77.6x**

Key Achievement: From 621 ms to 8 ms — **77x speedup** with zero accuracy loss

Future Optimization Opportunities

What Could Be Done Next

NPU Acceleration:

- Offload ResNet18 feature extraction
- Running CNN backbone on **NPU** to free up GPU

Mahalanobis Distance Optimization:

- Explore optimal versions compatible for **NPU/GPU**
- Custom kernels for einsum operations
- Fused operations to reduce memory bandwidth

Current Status:

- 125 FPS already exceeds real-time requirements
- Further optimization for edge deployment scenarios

Summary

77x Faster | 8ms Inference | 125 FPS

Metric	Baseline	Optimized
Time	621 ms	**8 ms**
FPS	1.6	**125**
Speedup	-	**77x**
Accuracy	90.5%	**90.5%**